

Shared Login & Employee Attribution

How a small pool of shared functional accounts, used by a much larger employee base, can still tell you exactly which person performed a given action — a login-time identity check, automatic per-message attribution, and opt-in deeper logging (including deletions) for individual models.

Module `hr_shared_login_binding` **Status** Working, verified end-to-end (see §8)

Scope `res.users` / `hr.employee` / `mail.thread` / `ir.model`

CONTENTS

1. The problem
2. Architecture: two independent pieces
3. Part 1 — Login-time identity verification
4. Part 2 — Message-level attribution (two layers)
5. Admin configuration: enabling deeper attribution for a model
6. The parentless-deletion log — `hr.attribution.log`
7. Consolidation: one note per batch, not one per record
8. Walkthrough (as tested)
9. Admin guide: marking an account as shared
10. Module reference
11. Design decisions and alternatives ruled out
12. Known limitations / open items
13. Full schema reference

1. The problem

A limited number of `res.users` accounts, a much larger number of `hr.employee` records. Many roles are shared responsibilities worked from a common login — e.g. two employees both use the **Supply Chain Planner**

account depending on shift. When a Purchase Order, Indent Request, or any other business record is created or changed, `create_uid` / `write_uid` only ever points at the shared account — never at the person who was actually at the keyboard.

The goal: know which employee performed a given action, without provisioning an individually licensed Odoo user for every employee.

2. Architecture: two independent pieces

The solution has two parts that work together but are independently useful:

PART	ANSWERS	WHERE IT LIVES
Login-time verification	"Which employee is behind this session, right now?"	<code>models/res_users.py</code> , <code>controllers/main.py</code>
Message-level attribution	"Tag every chatter message this session posts, on every model, with that employee — and optionally log the changes and deletions that don't naturally produce a message at all."	<code>models/message_attribution.py</code> , <code>models/attribution_config.py</code> , <code>models/attribution_log.py</code>

Neither part requires per-employee Odoo user licenses. Only the functional accounts are licensed `res.users`; employees prove their identity with their own existing personal login (the same one used for HR self-service today).

3. Part 1 — Login-time identity verification

Rather than building a custom session gate, this reuses the exact mechanism Odoo's own two-factor authentication (`auth_totp`) is built on:

- `res.users._mfa_type()` / `_mfa_url()` — hooks core already calls to ask "does this account need a second verification step, and if so, where do I send the user?"
- The session's `pre_uid` → `finalize()` flow (`odoo/http.py`) — when `_mfa_url()` returns a URL, the session is left in a pending state (`pre_uid` set, `uid` unset) until that second step calls `finalize()`. Every route that checks `session.uid` — nearly all of them — automatically treats a pending session as "not logged in," so no separate bypass-guard was needed.

What happens on login

1. A user signs in with a functional account's own credentials (e.g. `planner@harleys.com`) at the normal Odoo login screen — no change to that screen.
2. If that account is flagged `is_shared_login`, Odoo's own MFA redirect mechanism sends the browser to a second page: **Confirm your identity**.
3. The employee enters *their own* existing personal login (the same credential they already use for HR self-service / `employee_portal`) — not a new password.
4. That credential is checked via the same `res.users.authenticate()` core uses for the primary login (so brute-force cooldown applies here too), but it never becomes the session's `uid` — it's a proof of identity, not a second login. If the account has an `authorized_employee_ids` allow-list configured, the employee must also be on it.
5. On success, the *original* functional account's session is finalized, and the employee's id is stored in the session (`employee_binding_id`) for the rest of that browser session.

An individual (non-shared) login skips step 2 entirely — zero UX change for anyone using their own personal account.

4. Part 2 — Message-level attribution (two layers)

Every chatter-enabled model (anything using `mail.thread` or a variant) is extended once, in `models/message_attribution.py`, via `_inherit = 'mail.thread'`. Because Odoo composes that mixin into every model that uses it, this applies automatically to every current *and future* model — no per-model registration file, no manifest dependency on the owning module.

Layer 1 — always on

Overrides `_message_create()` — the single point every message-posting path converges on before a `mail.message` row is actually written, whether it came from:

- A manual "Log note" or Discuss message (`message_post()`),
- Odoo's own automatic field-change tracking (`_message_log()`) — this matters: a plain tracked-field change with no custom subtype configured, like a status change on an Indent, does *not* go through `message_post()` at all — an earlier version of this module missed these until the hook was moved to `_message_create()`,
- Or any explicit business-logic call to post a message.

Adds `employee_id` to the message (structured and queryable, not just text) and appends an inline "*— Logged by <employee>*" note. Free, automatic, no configuration.

SAFETY PROPERTIES

Wrapped in try/except — attribution can never break the underlying business action (confirming a PO, saving a record) it's attached to, only fail silently with a logged warning. No recursion risk (`mail.message` doesn't itself inherit `mail.thread`). No new noise — it only enriches messages that were already going to be posted; it never manufactures new ones.

Layer 2 — opt-in per model

Layer 1 only covers what already gets posted as a chatter message. Two real gaps remain structurally out of its reach:

- **Fields with no `tracking=True` configured** — a plain field change nobody asked Odoo to log in the first place.
- **Line items** — child records such as Indent Request Lines, whose own chatter panel isn't shown anywhere in the UI.

Layer 2 closes both, but only where an admin opts a specific model in. It overrides `create()`/`write()`/`unlink()` on the same class, gated by three new fields on `ir.model` (see §5). Where a record has a parent document, the note is posted on the *parent's* chatter instead of the child's invisible one — see §5 for exactly how the parent is identified, and §7 for how batches of records are consolidated into a single note instead of one per row.

5. Admin configuration: enabling deeper attribution for a model

Layer 2 reuses **Settings** → **Technical** → **Models** (`ir.model`) rather than a new screen — already searchable by name/module, already technical-user-only. A new **Employee Attribution** tab (restricted to `base.group_system`) adds:

FIELD	TYPE	MEANING
<code>employee_attribution_log_changes</code>	Boolean	Post an explicit "Created/Updated" note on every create/write for this model, regardless of field tracking. Off by default.

FIELD	TYPE	MEANING
<code>employee_attribution_log_deletions</code>	Boolean	Post a "Deleted" note when a record is removed. Off by default.
<code>employee_attribution_parent_field_id</code>	Many2one → <code>ir.model.fields</code>	The Many2one field on <i>this</i> model pointing to its parent document (e.g. "Indent Request" on Indent Request Line). A dropdown domain-filtered to Many2one fields belonging to the model being configured — not a technical name typed by hand.

A constraint (`_check_employee_attribution_config`) blocks enabling either boolean on:

- Technical/infrastructure models (`mail.*`, `bus.*`, `ir.*`, `base_automation`, `iap.*`, `studio.*`, `res.users`) — hard-excluded regardless of admin intent, to avoid recursion/noise in the mail/system internals.
- Any model without a chatter to log to (a `message_post` capability check).
- A `parent_field_id` that doesn't actually belong to the model, or isn't a Many2one.

WHAT "EMPTY PARENT FIELD" MEANS — THIS DIFFERS BY ACTION

Create/Update with no parent field → note posts on the record's *own* chatter. Correct default for top-level documents.

Delete with no parent field → does *not* post on the record (it would be wiped by `mail.thread.unlink()`'s own message cleanup moments later) — falls back to the standalone log table instead (§6).

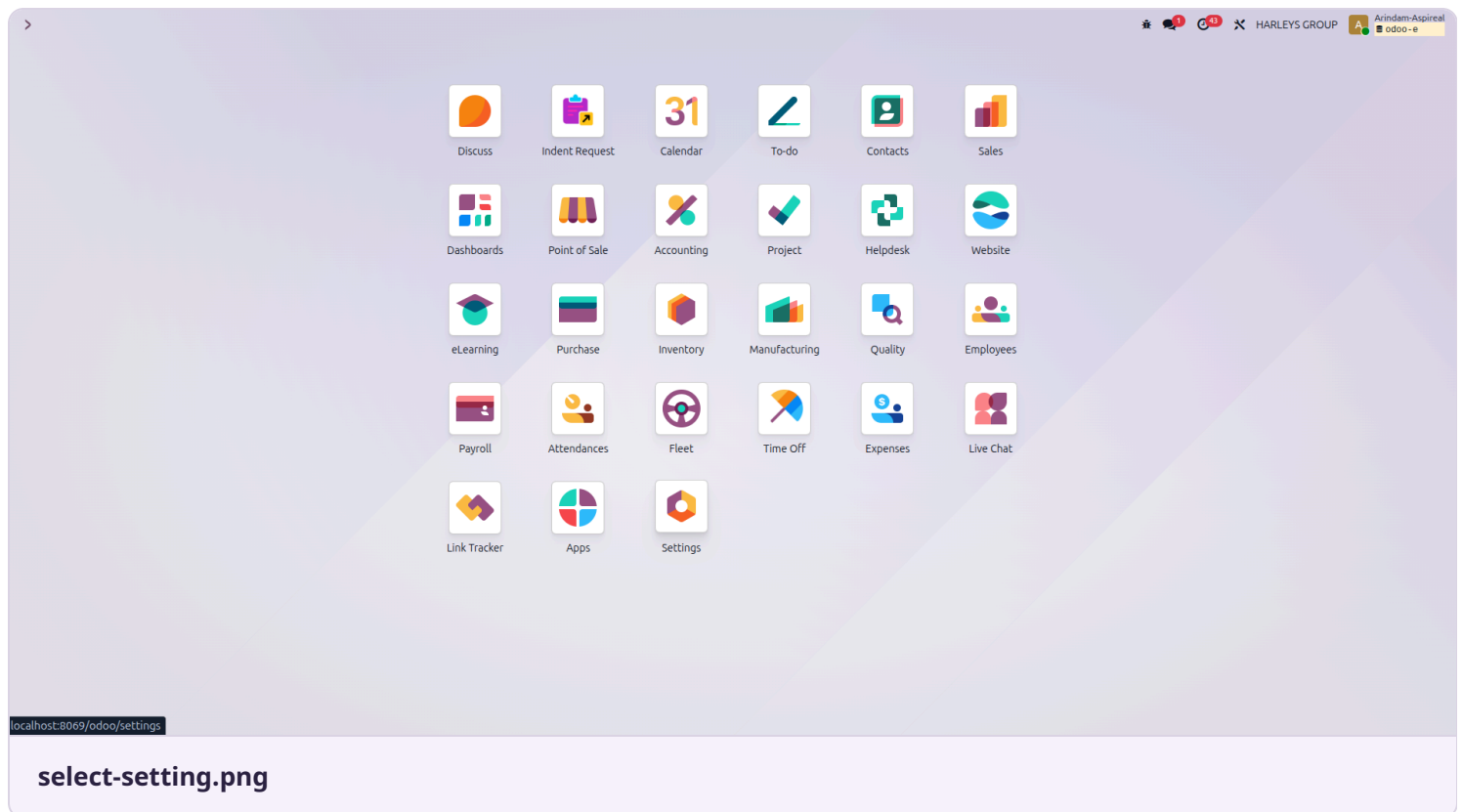
Auto-detecting the parent field

`_auto_detect_attribution_parent_field()` uses a heuristic: the Many2one field with `onDelete='cascade'` is the standard Odoo convention for a line item's relation to its owning document (`sale.order.line.order_id`, `account.move.line.move_id`, `stock.move.line.picking_id`, this repo's own `indent.request.line.request_id` all use it). It returns that field when there's exactly one candidate, and returns nothing — left for manual review — when there are zero or multiple candidates. Genuinely top-level documents correctly yield nothing, since they don't have one.

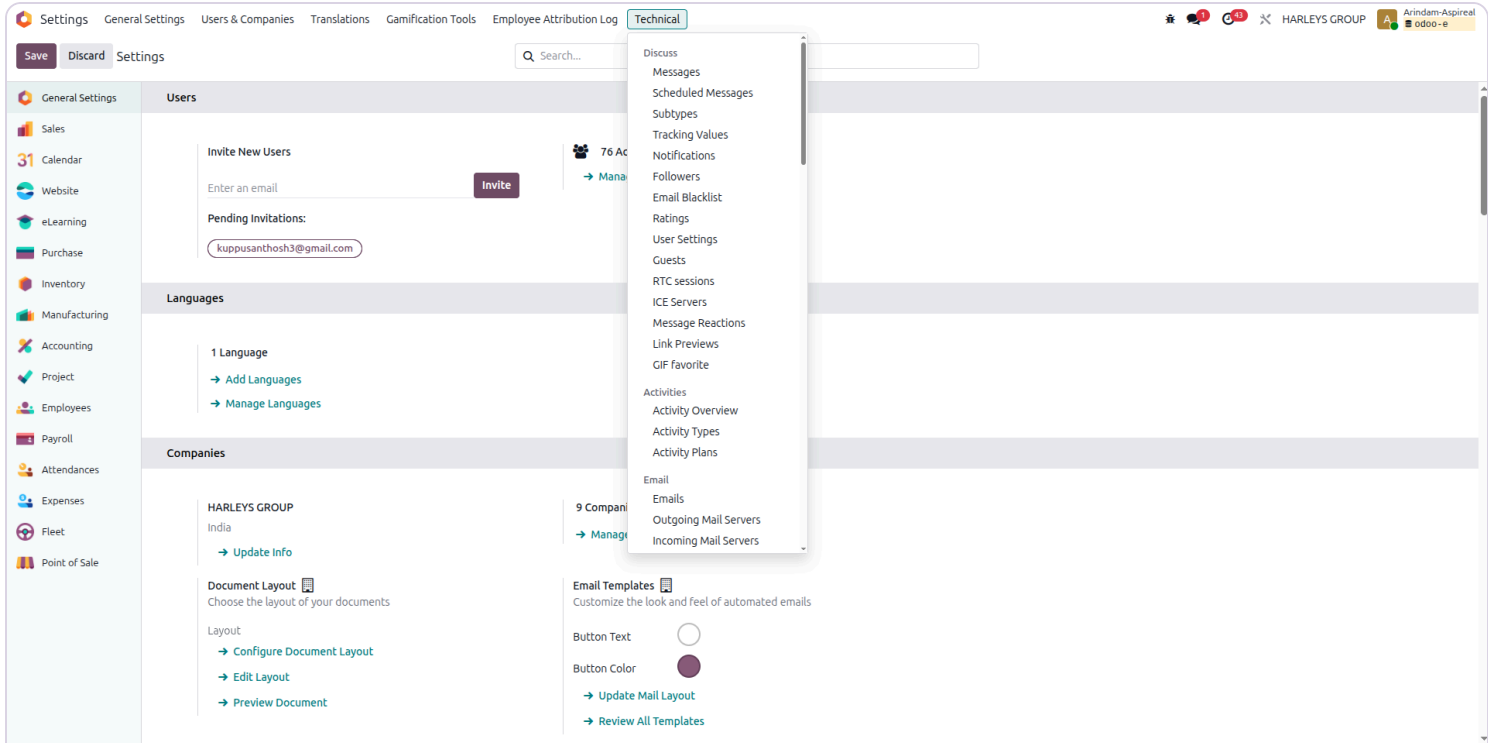
Bulk "Enable Employee Attribution"

`action_enable_employee_attribution()` is a bulk operation, exposed as a contextual action in the Models list view: select any rows — including all of them — and run it. For each selected row, it skips (and logs) anything technical/incompatible; otherwise it turns both booleans on and auto-fills the parent field if one isn't already set and the heuristic finds exactly one candidate. This was run once already across the instance's model list.

1 Open Settings

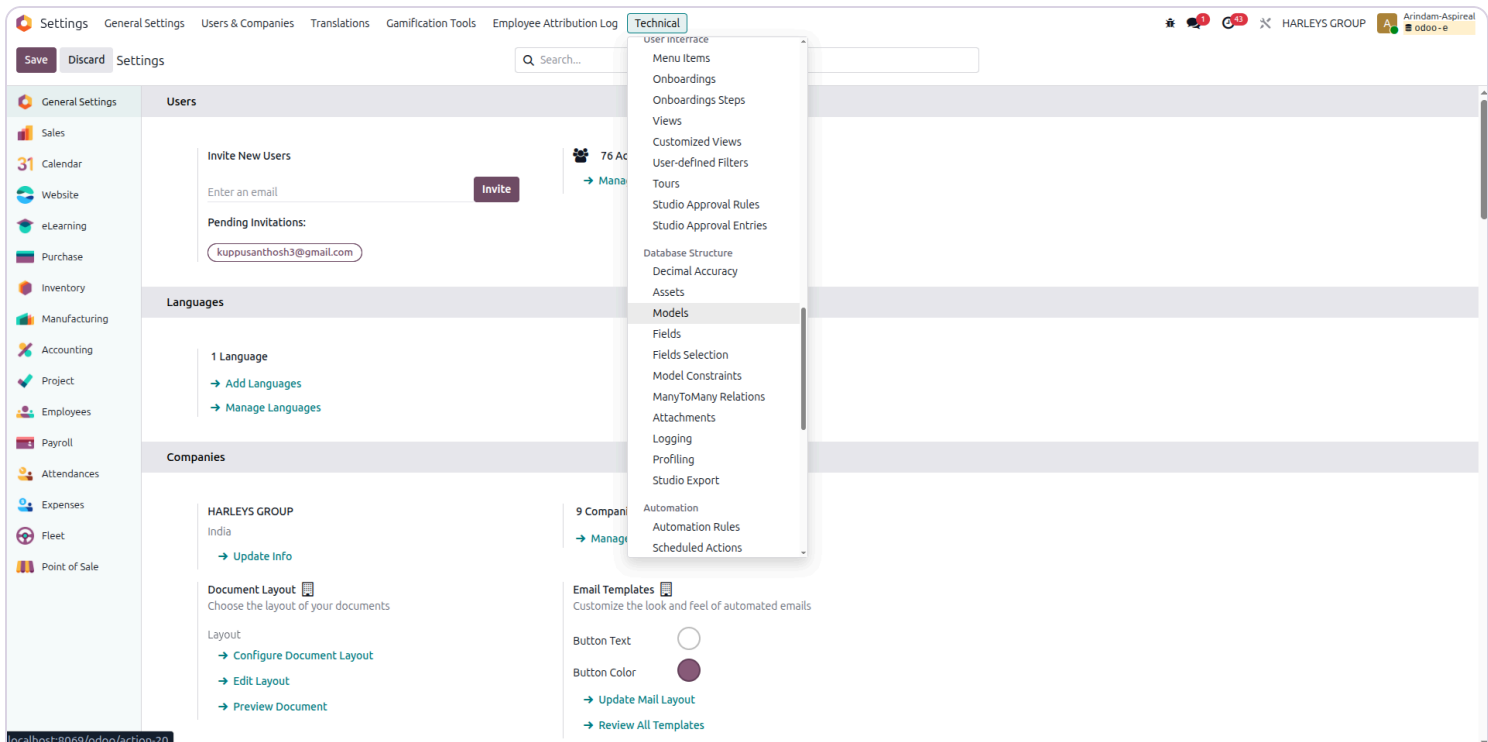


2 Switch to the Technical tab



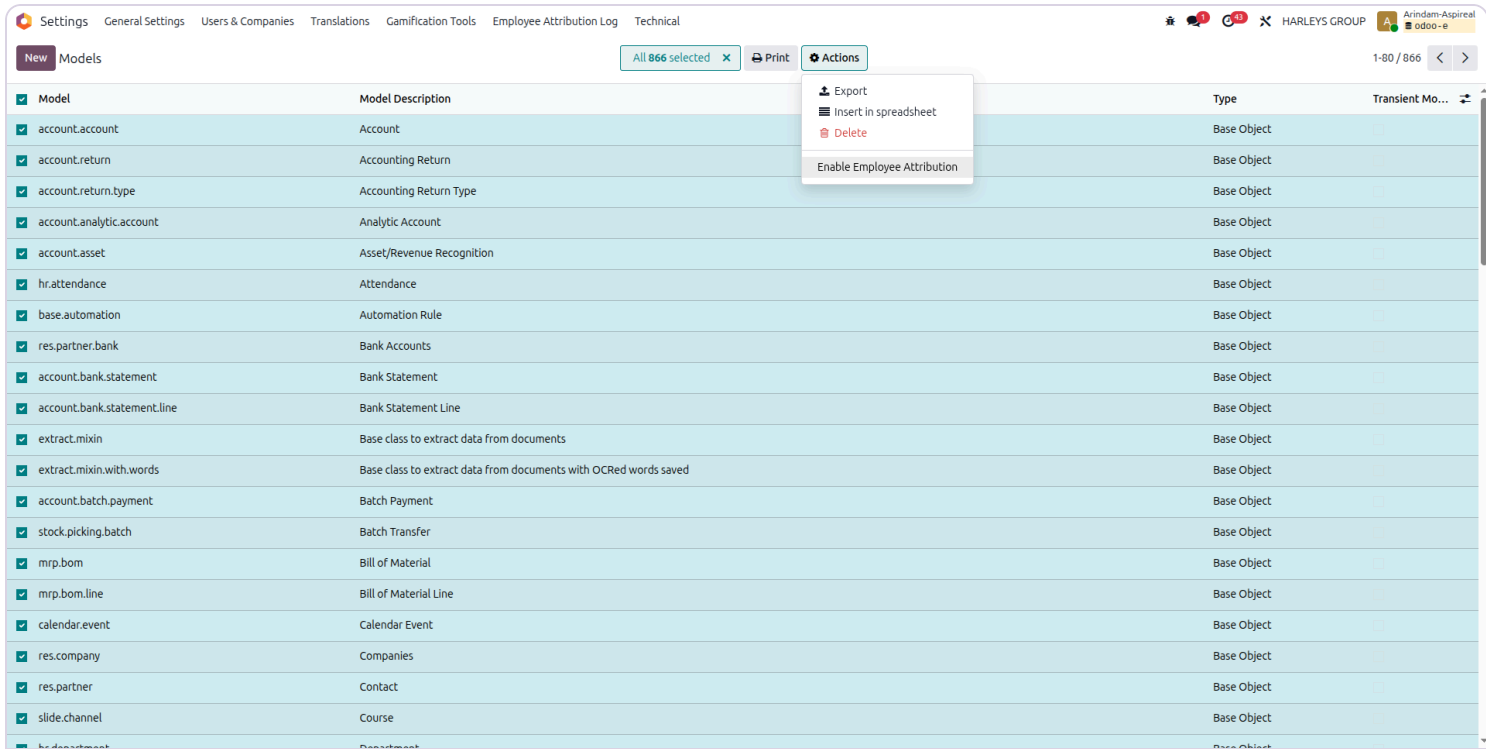
go_to_technical_tab.png — note the new **Employee Attribution Log** entry already in the top menu bar (§6).

3 Open Models under Database Structure



select_models.png

4 Bulk-enable across every model, or just the ones you pick



The screenshot shows the Odoo 'Models' configuration page. At the top, a navigation bar includes 'Settings', 'General Settings', 'Users & Companies', 'Translations', 'Gamification Tools', 'Employee Attribution Log', and 'Technical'. On the right, there are notification icons, the company name 'HARLEYS GROUP', and the user 'Arindam-Aspireal'. Below the navigation bar, a header bar shows 'New Models', a status 'All 866 selected', a 'Print' button, and an 'Actions' button. The main table lists various models with columns for 'Model', 'Model Description', 'Type', and 'Transient Mo...'. All rows are checked, indicating they are selected. An 'Actions' dropdown menu is open, showing options: 'Export', 'Insert in spreadsheet', 'Delete', and 'Enable Employee Attribution'. The table contains the following data:

Model	Model Description	Type	Transient Mo...
account.account	Account	Base Object	☐
account.return	Accounting Return	Base Object	☐
account.return.type	Accounting Return Type	Base Object	☐
account.analytic.account	Analytic Account	Base Object	☐
account.asset	Asset/Revenue Recognition	Base Object	☐
hr.attendance	Attendance	Base Object	☐
base.automation	Automation Rule	Base Object	☐
res.partner.bank	Bank Accounts	Base Object	☐
account.bank.statement	Bank Statement	Base Object	☐
account.bank.statement.line	Bank Statement Line	Base Object	☐
extract.mixin	Base class to extract data from documents	Base Object	☐
extract.mixin.with.words	Base class to extract data from documents with OCRd words saved	Base Object	☐
account.batch.payment	Batch Payment	Base Object	☐
stock.picking.batch	Batch Transfer	Base Object	☐
mrp.bom	Bill of Material	Base Object	☐
mrp.bom.line	Bill of Material Line	Base Object	☐
calendar.event	Calendar Event	Base Object	☐
res.company	Companies	Base Object	☐
res.partner	Contact	Base Object	☐
slide.channel	Course	Base Object	☐
hr.department	Department	Base Object	☐

selectall_models_and_enable_employee_attribution.png — select the rows to configure (here, every model in the registry) → **Actions** → **Enable Employee Attribution**. Incompatible rows are skipped automatically.

5 Fine-tune a single model by hand

SettingsGeneral SettingsUsers & CompaniesTranslationsGamification ToolsEmployee Attribution LogTechnical

NewModelsIndent Request

1 / 8<>

Model Description ?
Model ?
Order ?
Abstract Model ?
Transient Model ?
Has Mail Thread ?
Has Mail Activity ?
Has Mail Blacklist ?

Indent Request
indent.request
id
☐
☐
☒
☒
☐

Type ?
Base Object
In Apps ?
harleys_customization

FieldsAccess RightsRecord RulesNotesEmployee AttributionViewsWebsite Forms

Log Every Change (Employee) ?
Log Deletions (Employee) ?
Attribution Parent Field ?

☒
☒
Leave blank for top-level documents

individual_model_employeeattribution.png — the same three fields from the table above, on a single model's record. Indent Request is a top-level document, so its parent field is correctly left blank.

6. The parentless-deletion log — `hr.attribution.log`

The one thing chatter genuinely cannot do: log a deletion for a record with **no** parent to redirect to (a top-level document, or a specific record whose parent field happens to be unset). Deliberately narrow in scope — this is not a general duplicate log, it exists only for that one gap. It is not `mail.thread`-based (it *is* the log; logging to it would be circular).

COLUMN	TYPE
<code>employee_id</code>	FK → <code>hr.employee</code>
<code>user_id</code>	FK → <code>res.users</code>
<code>company_id</code>	FK → <code>res.company</code>
<code>res_model</code>	Char
<code>res_id</code>	Integer

COLUMN	TYPE
<code>record_name</code>	Char
<code>action</code>	Selection (currently only <code>'deleted'</code>)
<code>logged_at</code>	Datetime

`_log_access = False` — no `create_uid` / `write_uid` / `create_date` / `write_date`, since `user_id` + `logged_at` already cover that, deliberately.

Read-only in the UI (`perm_write` / `perm_create` / `perm_unlink` all `0`) for `base.group_system` and `hr.group_hr_manager`; only ever written via `sudo()` from the deletion fallback path in code. Browsable at **Settings** → **Administration** → **Employee Attribution Log**.

7. Consolidation: one note per batch, not one per record

`_log_attribution_change()` and `_log_attribution_deletion()` group records by parent rather than looping and posting per record. Verified against this repo's actual code — both noisy cases reported during testing are each a single batched ORM call, so both are already fully consolidated by this grouping (one summary note like "Created: 40 line(s)." per parent per call, or the specific description when only one record is involved):

- A template loading many `indent.request.line` records via `(0,0,vals)` One2many commands.
- `indent_request.py`'s `request.line_ids.write({'state': 'sent'})` status change.

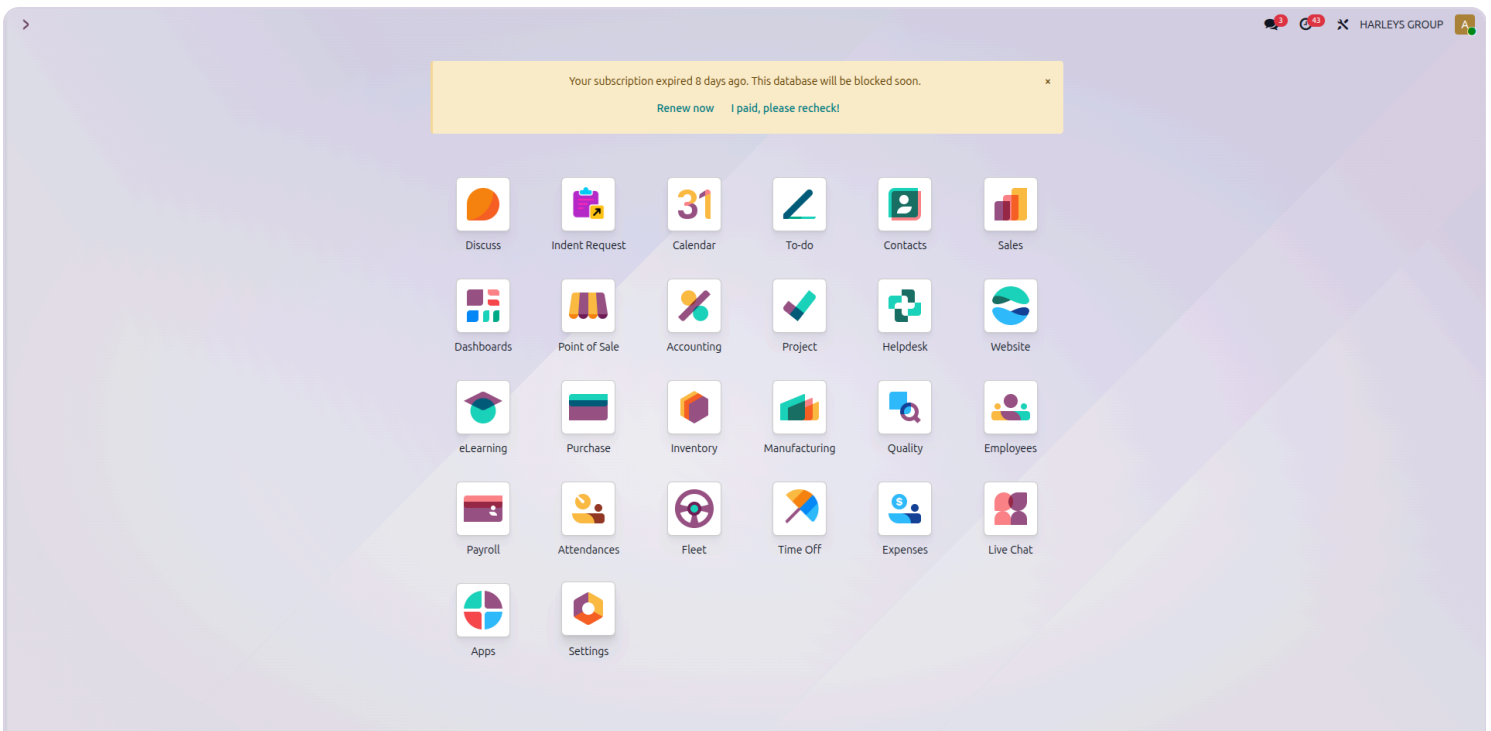
BOUNDARY WORTH KNOWING

Consolidation only works within a single ORM call. Code that creates/updates line items one at a time in a Python loop (none currently does, for the cases above) would still produce one message per iteration, since each is a separate call from the ORM's point of view.

8. Walkthrough (as tested)

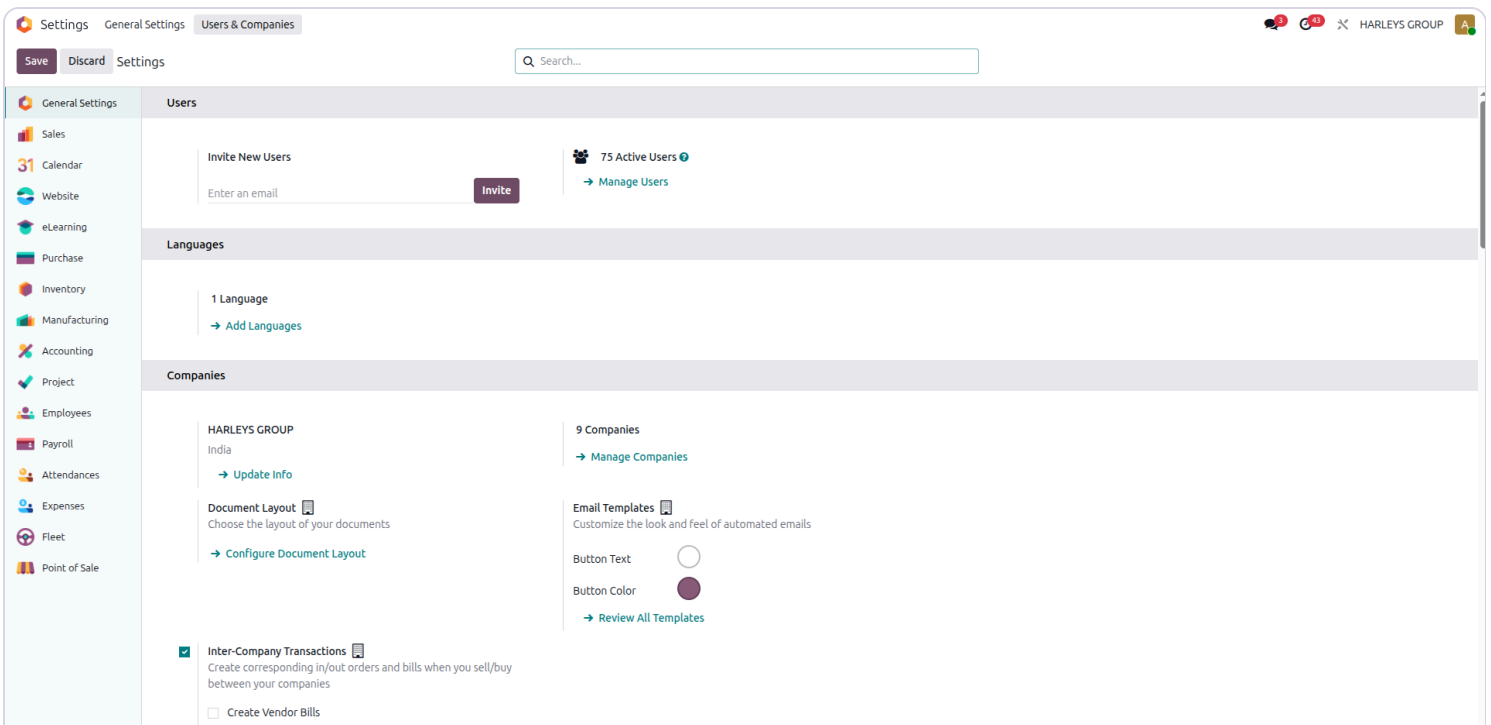
This is the exact sequence used to verify the login-time verification and Layer 1 attribution, using the **Supply Chain Planner** shared account and two real employees.

1 Confirm the account before setup

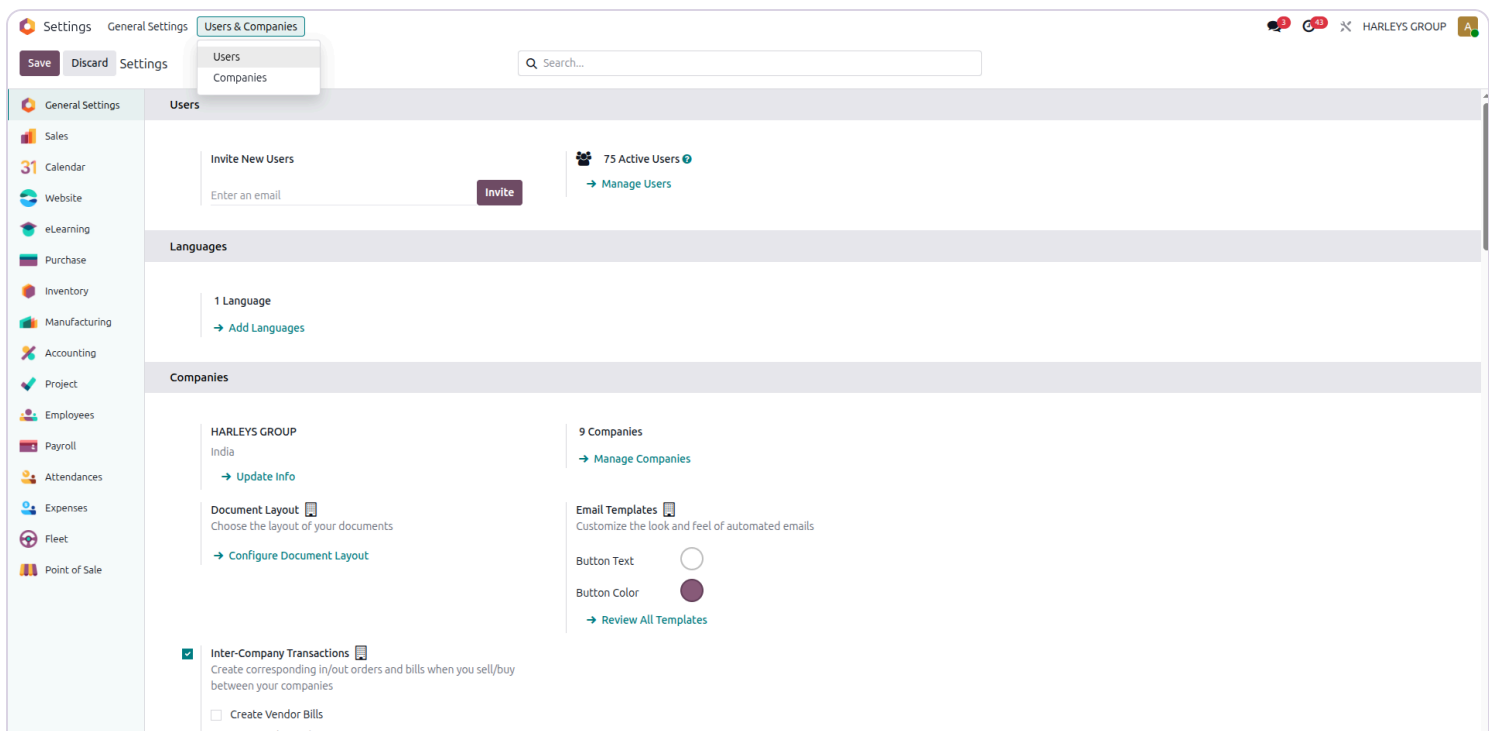


apps-launcher.png — the standard Odoo home screen, for reference / orientation.

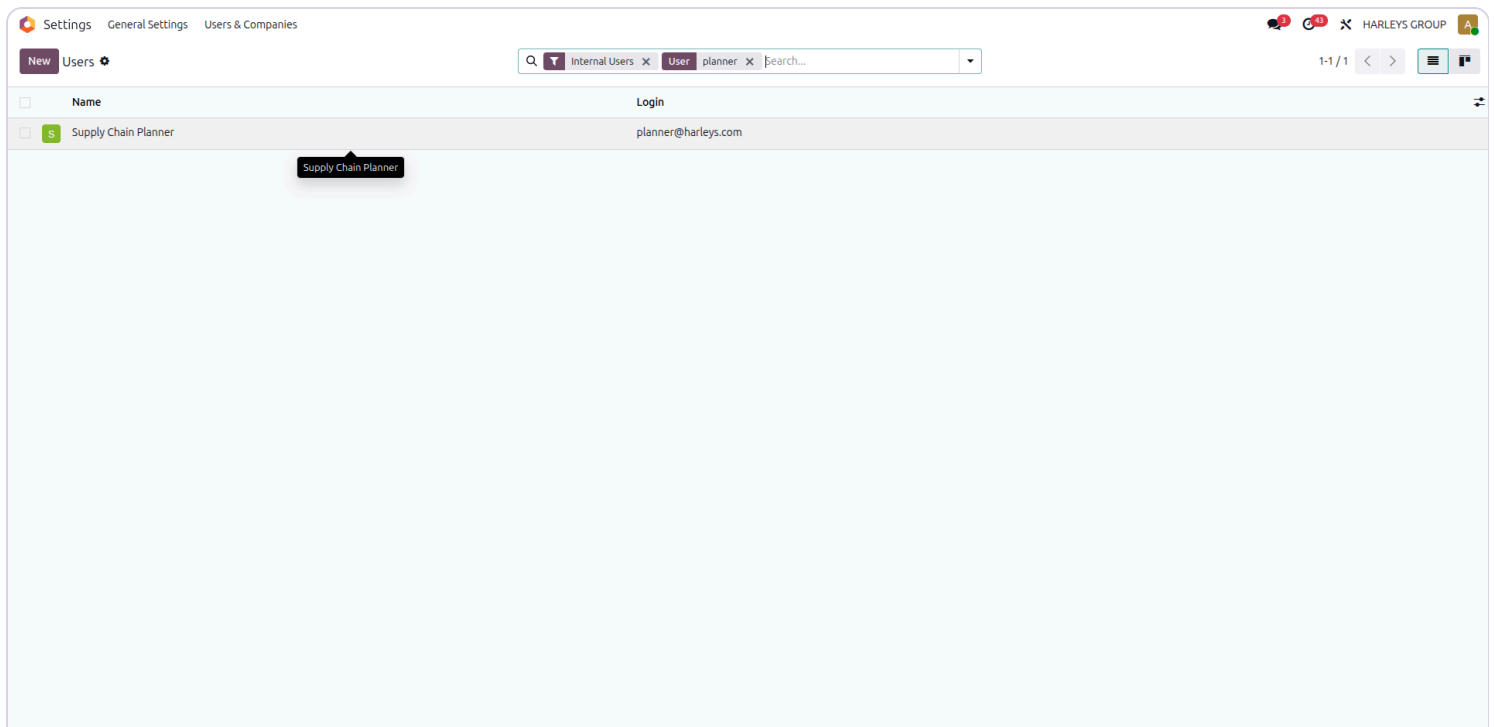
2 Locate the account in Settings



settings-general.png — confirms the licensed-user baseline this feature works within.

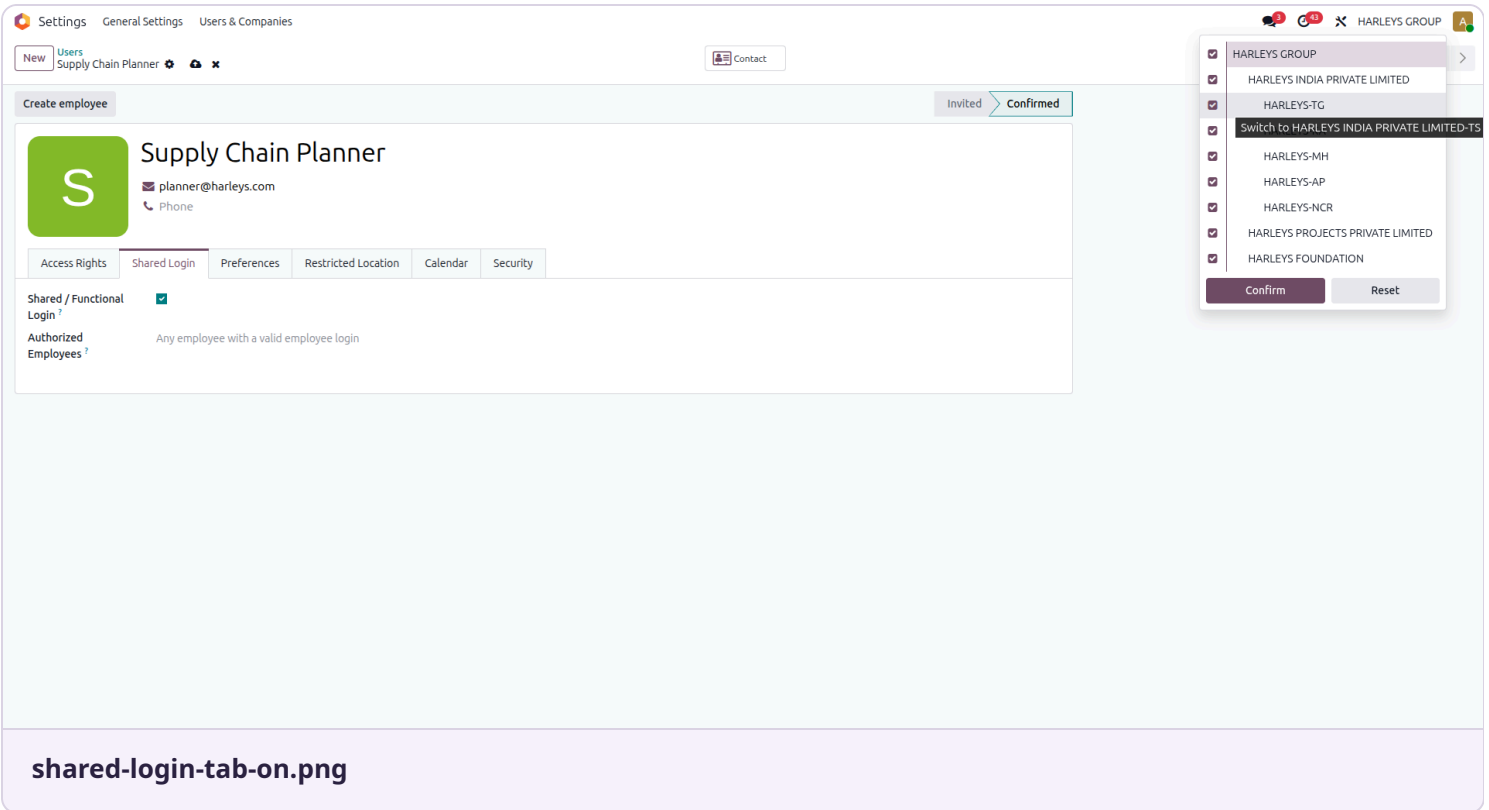
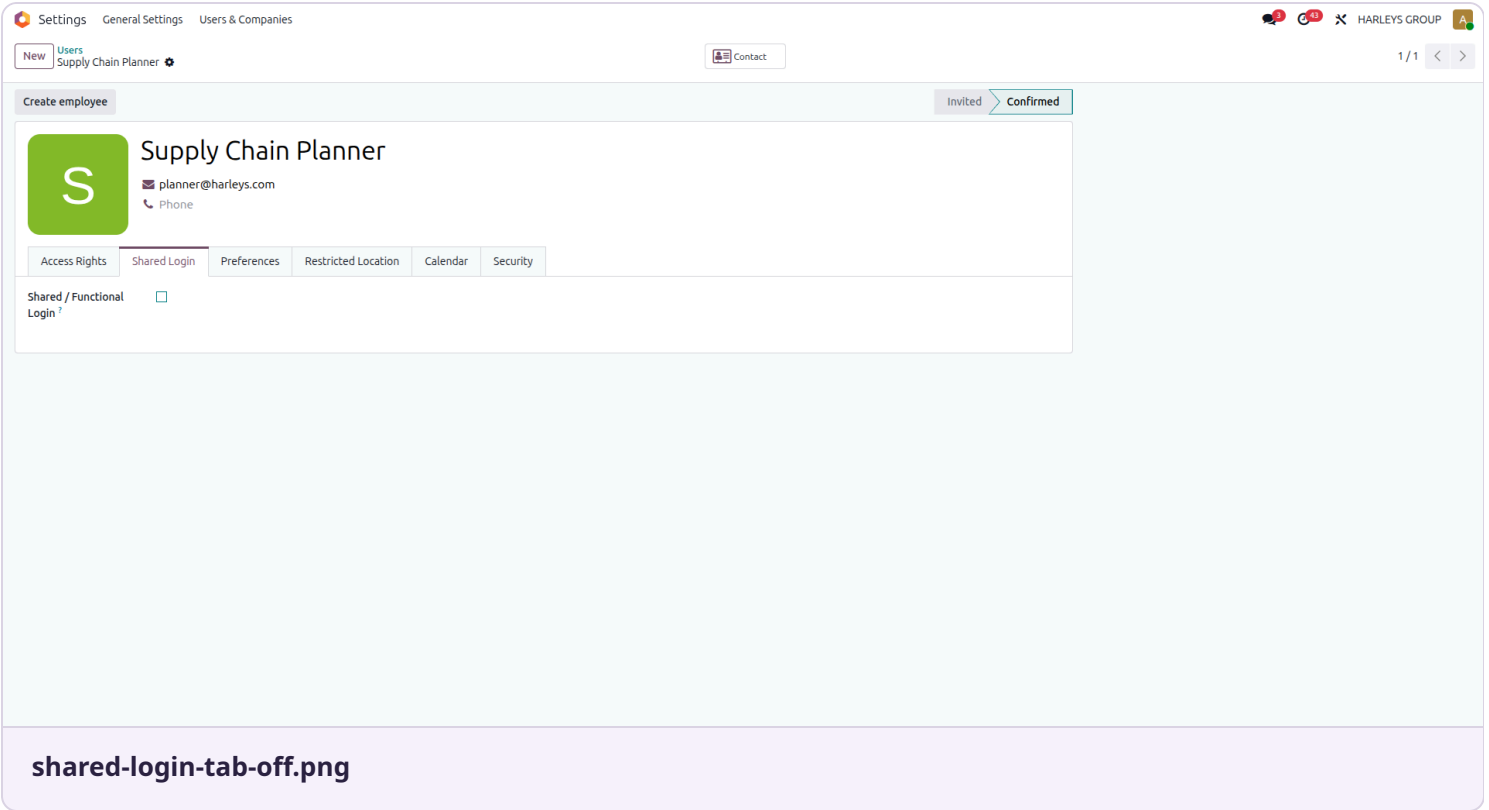


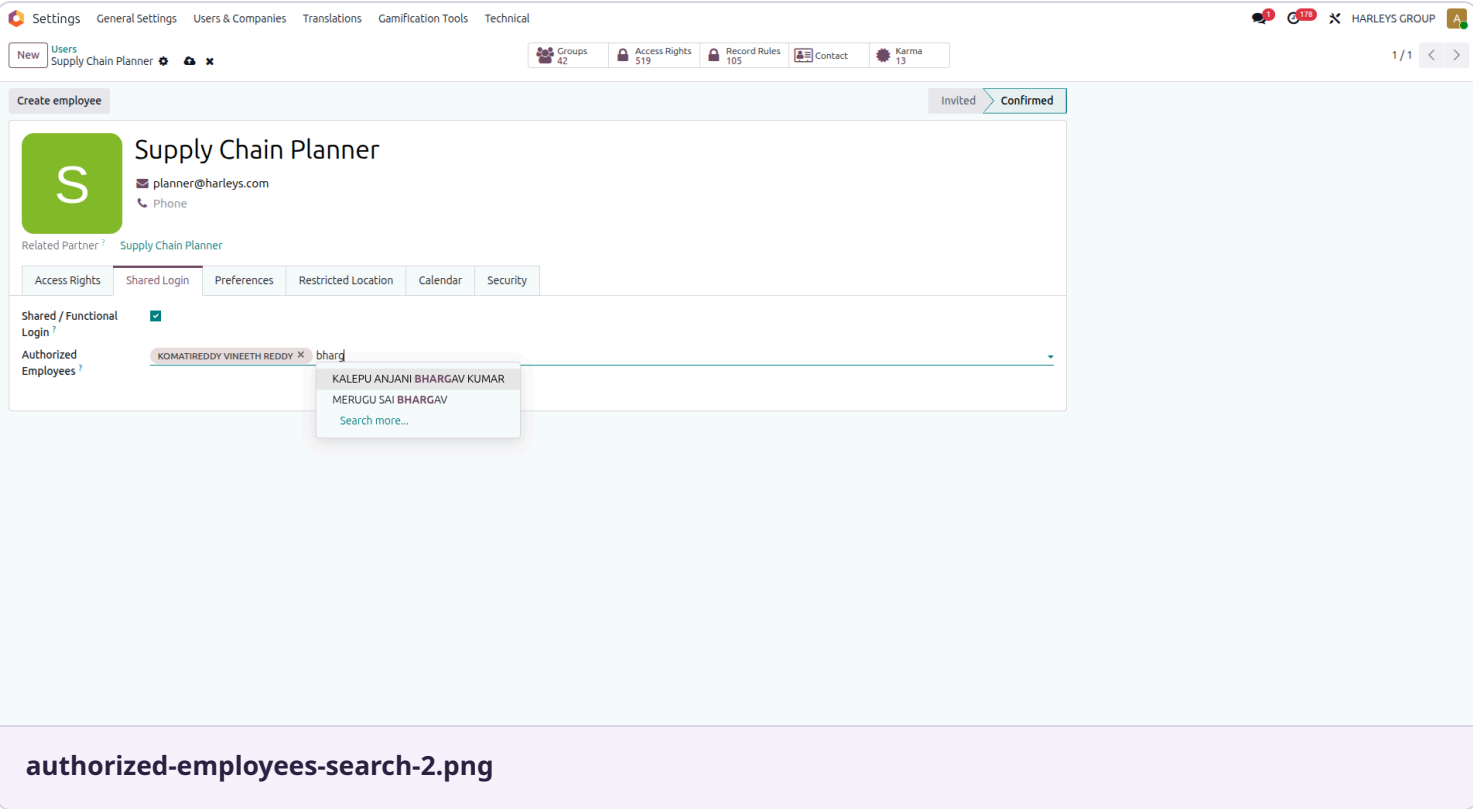
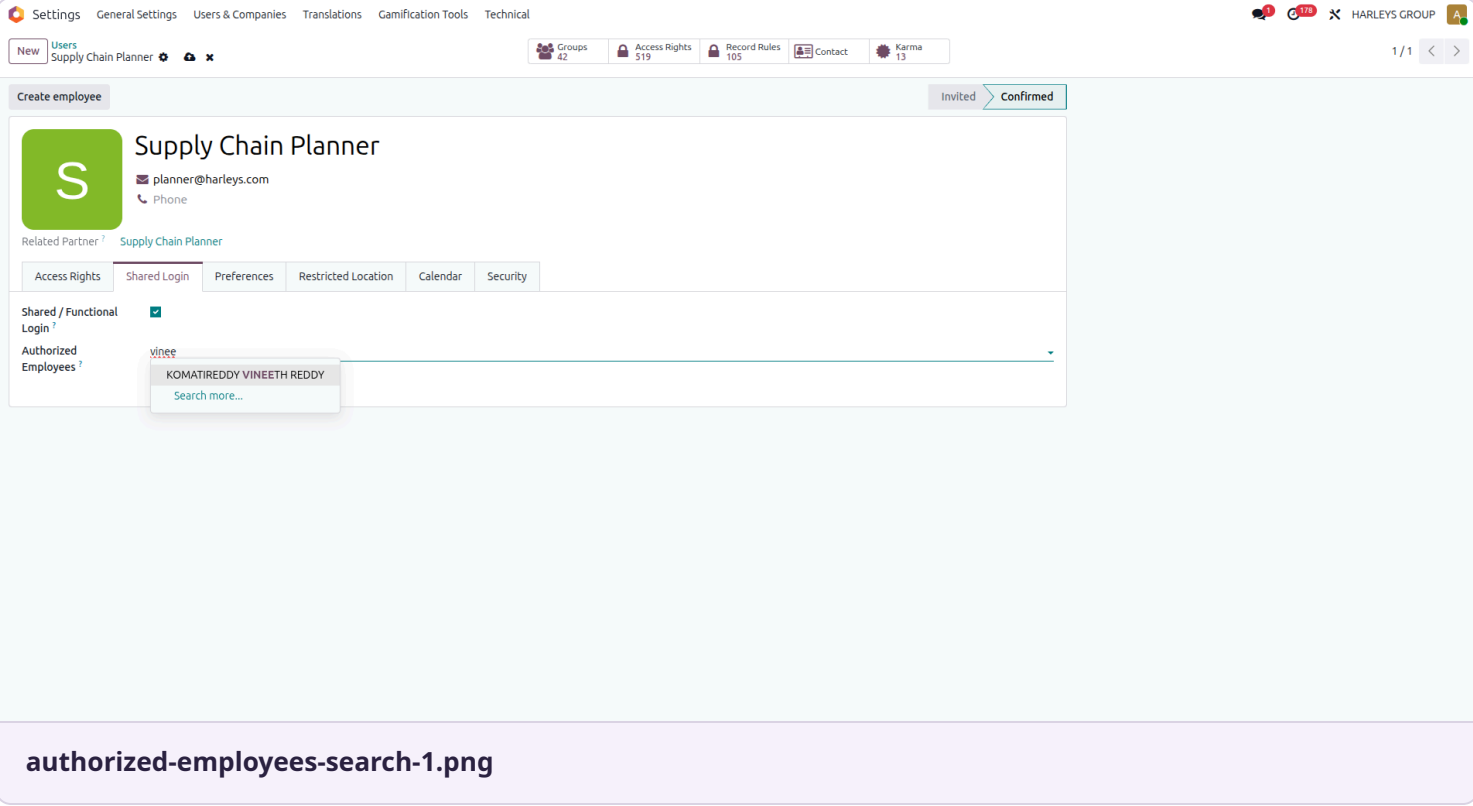
settings-users-menu.png



users-list-planner.png

3 Mark the account as shared and authorize employees





SettingsGeneral SettingsUsers & CompaniesTranslationsGamification ToolsTechnical

NewUsersSupply Chain Planner

Groups 42Access Rights 519Record Rules 105ContactKarma 13

1 / 1

Save manually

Create employeeInvitedConfirmed

S

Supply Chain Planner

planner@harleys.com

Phone

Related Partner?Supply Chain Planner

Access RightsShared LoginPreferencesRestricted LocationCalendarSecurity

Shared / Functional Login?☒

Authorized Employees?

KOMATIREDDY VINEETH REDDY XKALEPU ANJANI BHARGAV KUMAR X

authorized-employees-saved.png — Komatireddy Vineeth Reddy and Kalepu Anjani Bhargav Kumar are now the only two employees permitted to bind their identity to this account.

4 First employee logs in through the shared account

HomeCoursesTickets

Sign in

EmailChoose a user

planner@harleys.com

PasswordReset Password

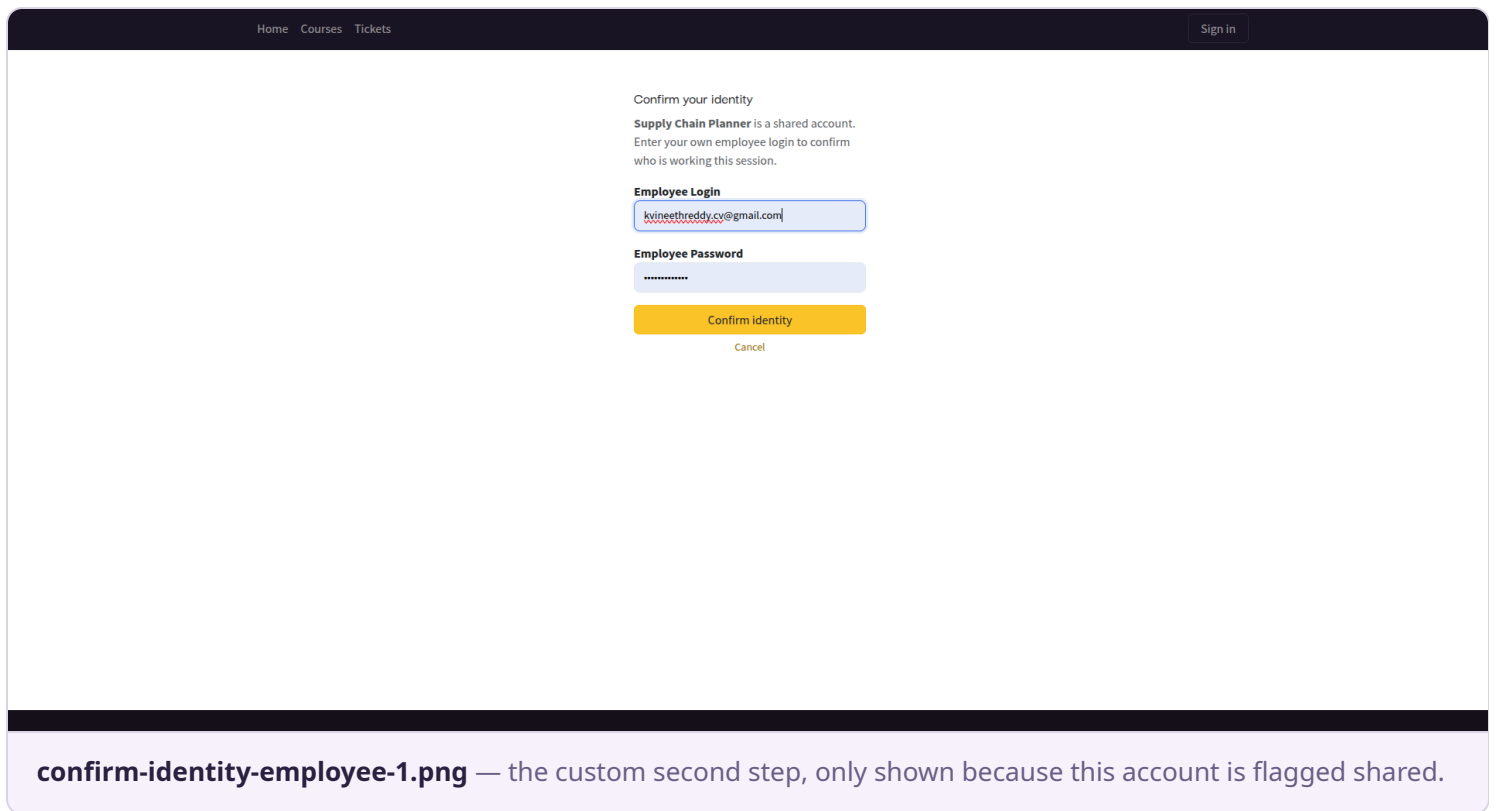
Log in

Log in as superuser

- or -

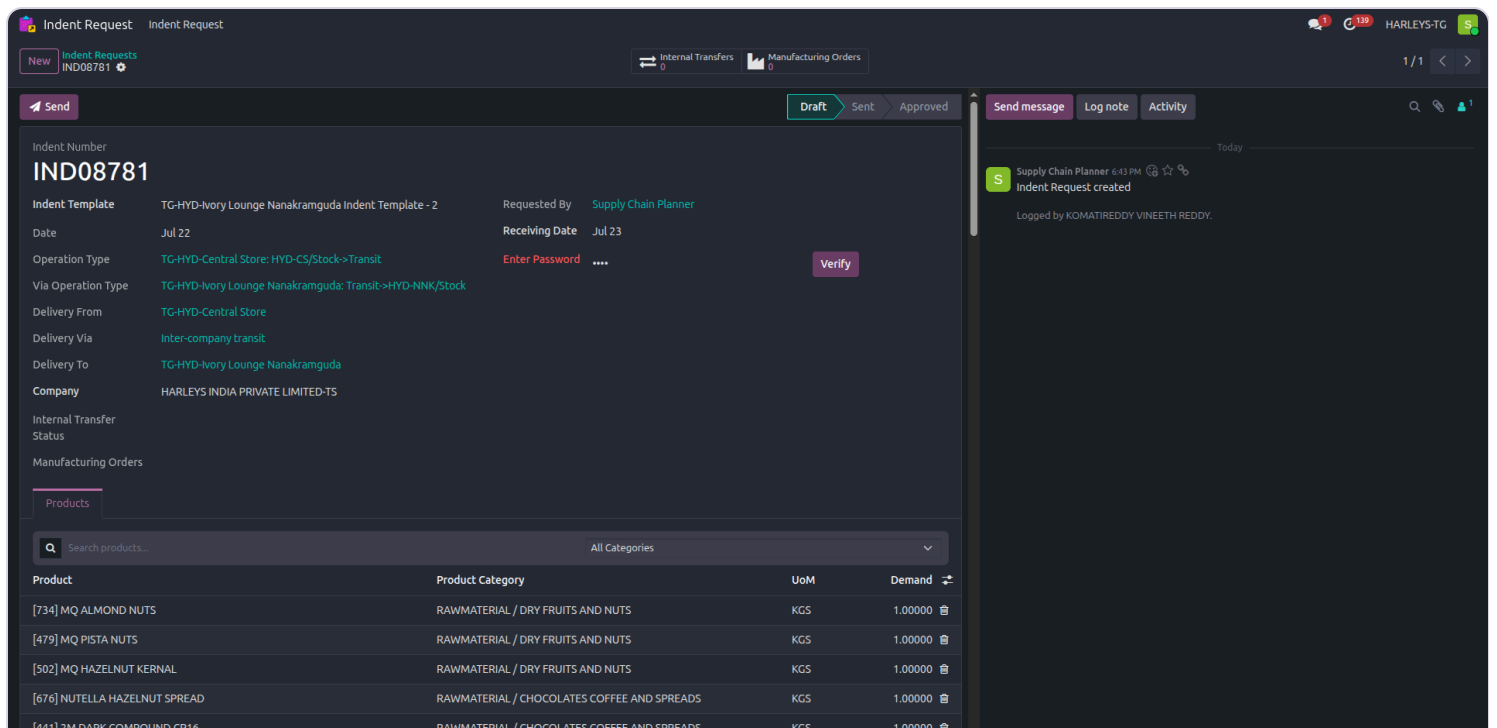
Use a Passkey

login-planner.png



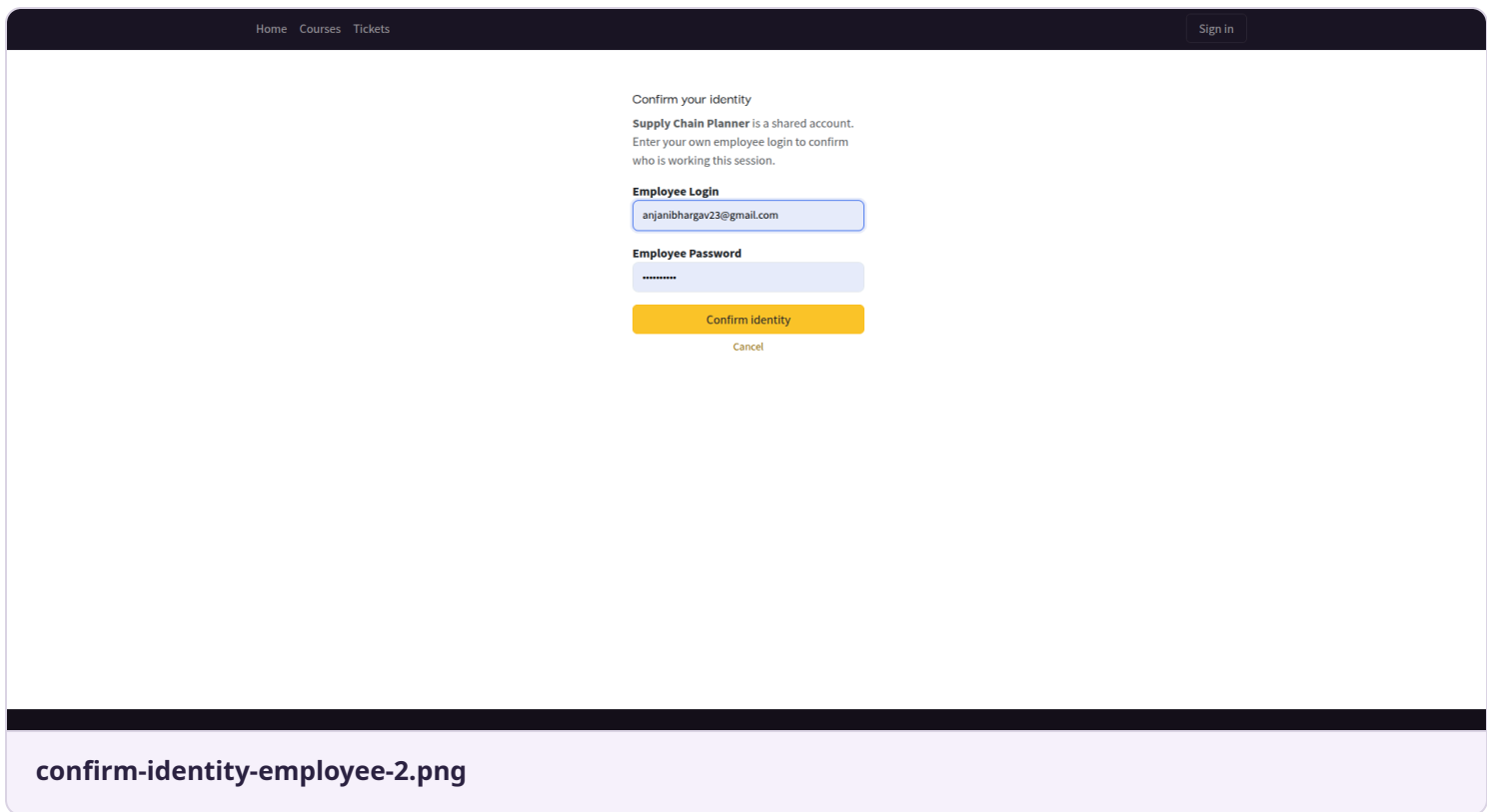
confirm-identity-employee-1.png — the custom second step, only shown because this account is flagged shared.

5 Create an Indent Request and check the chatter

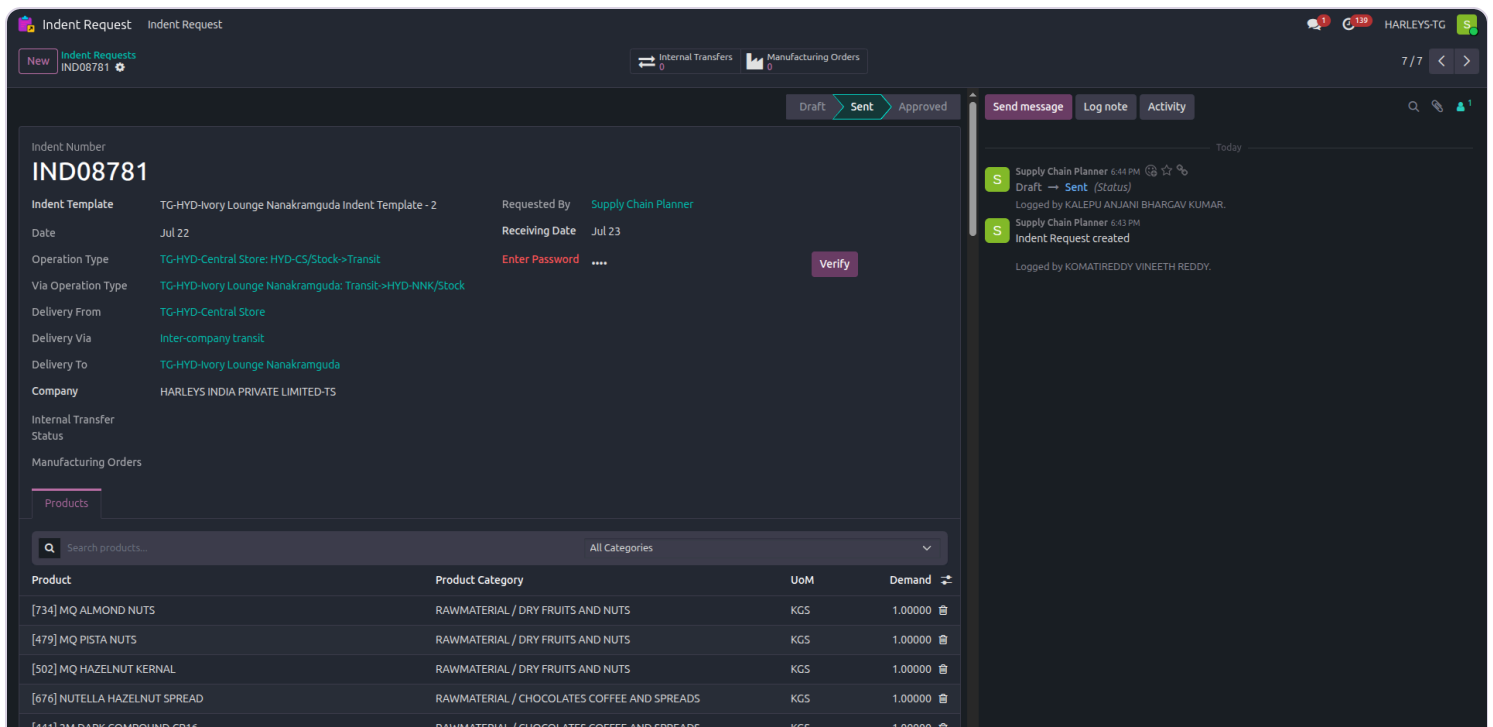


indent-created-tagged.png — first proof point: the creation message, which goes through Odoo's no-subtype tracking path, carries the correct employee.

6 Log out, second employee logs in through the same shared account



7 Change the record's status and check the chatter again



indent-status-change-tagged.png — the decisive proof: *two different employees, sharing one login*, get correctly and separately attributed for two different actions on the same record.

9. Admin guide: marking an account as shared

1. Settings → Users & Companies → Users, open the account.
2. Open the **Shared Login** tab.
3. Check **Shared / Functional Login**.
4. Optionally add specific employees to **Authorized Employees**. Leave empty to allow any employee with a valid personal login to bind to this account; populate it to restrict which employees may use it.
5. Save.

No restart or module update is needed for this step — it's a plain data change. The **Shared Login** tab itself is only visible to users with Settings/Administration access (`base.group_system`).

10. Module reference — `hr_shared_login_binding`

FILE	RESPONSIBILITY
<code>__manifest__.py</code>	Module metadata and dependencies.
<code>models/res_users.py</code>	<code>is_shared_login</code> , <code>authorized_employee_ids</code> fields; <code>_mfa_type()</code> / <code>_mfa_url()</code> overrides that trigger the employee-verify step, chaining through <code>super()</code> so an account with real 2FA configured keeps using that instead.
<code>controllers/main.py</code>	The <code>/web/login/employee_verify</code> route: renders the second-step form, validates the submitted employee credential, checks <code>authorized_employee_ids</code> if set, finalizes the functional account's session and binds the employee id to it.
<code>views/employee_verify_templates.xml</code>	The "Confirm your identity" page template, built on the same layout as the standard login and 2FA pages.
<code>views/res_users_views.xml</code>	Adds the Shared Login tab to the user form (admin-only).
<code>models/message_attribution.py</code>	Both attribution layers on the same <code>mail.thread</code> -extended class: the Layer 1 <code>_message_create()</code> override (<code>employee_id</code> field + "Logged by" note), and the Layer 2

FILE	RESPONSIBILITY
	<code>create()</code> / <code>write()</code> / <code>unlink()</code> overrides plus the consolidation helpers (§7), gated by the <code>ir.model</code> config below.
<code>models/attribution_config.py</code>	The <code>ir.model</code> extension: the three Employee Attribution fields (§5), the compatibility constraint, the parent-field auto-detect heuristic, and the bulk "Enable Employee Attribution" action.
<code>models/attribution_log.py</code>	<code>hr.attribution.log</code> (§6) — the fallback log for parentless deletions.
<code>views/ir_model_views.xml</code>	The Employee Attribution tab on the model record, and the bulk action entry in the Models list view.
<code>views/attribution_log_views.xml</code>	Read-only list/form views for <code>hr.attribution.log</code> and its Settings → Administration menu item.
<code>models/employee_attribution_mixin.py</code> , <code>models/attribution_targets.py</code>	An earlier, per-model create/write implementation. Disabled (not imported) but kept on disk as a fallback/reference — see §11.

11. Design decisions and alternatives ruled out

Several approaches were evaluated before arriving at the current design:

- **PIN/badge gate modeled on `mrp_workorder`'s Shop Floor login** — rejected as the primary mechanism because it only covers interactive UI sessions, not API/automation-triggered writes, and adds friction to every single action rather than once per session.
- **Resolving identity from `hr_attendance` clock-in/out data** — rejected as unreliable: stale "still checked in" state and simultaneous multi-employee attendance would create ambiguous attribution. Login-time verification is deterministic instead.
- **A brand-new password field on `hr.employee`** — rejected in favor of reusing each employee's existing personal login (already provisioned via `employee_portal`), avoiding a second credential store and, importantly, avoiding any new licensed `res.users` seats.
- **Per-model `create` / `write` overrides, explicitly listed (`attribution_targets.py`)** — the first working implementation; scaled to dozens of models across many module dependencies before being judged too heavy

to maintain and test. *Layer 2 (§4, §5) later revisits create/write overrides, but as one generic, config-driven mechanism gated by `ir.model` fields — not one Python class per model.*

- **A blanket, fully-automatic patch across every model in the registry** — considered and rejected: real recursion risk (a message-posting patch could trigger itself via `mail.message` creation if not scoped carefully), noise on high-frequency internal writes, and harder to debug than an explicit mechanism. Layer 2's opt-in-per-model design and the technical/infrastructure exclusion list (§5) are the direct answer to this concern.
- **The current approach — one hook on `mail.thread._message_create()`, plus opt-in `create` / `write` / `unlink` overrides on the same class** — adopted because Layer 1 only enriches messages that were already going to be posted (no new noise, no recursion risk, since `mail.message` doesn't inherit `mail.thread`) and applies automatically everywhere, while Layer 2 closes the remaining gaps (untracked fields, line items, deletions) only where an admin explicitly opts a model in.

12. Known limitations / open items

- **Enabling `employee_attribution_log_changes` on a model that already has `tracking=True` fields** (most real business documents) produces a second, redundant note alongside the native tracking message Layer 1 already tags. The bulk-enable action does not currently avoid this — it turns both flags on regardless of whether native tracking already covers a model. A more surgical default (only auto-enable on models with no tracked fields, or with a parent field) was discussed as an alternative but not built.
- **The auto-detect heuristic is a best guess, not a guarantee** — models with more than one `ondelete='cascade'` Many2one are left for manual review, and it cannot distinguish "genuine top-level document" from "child model that just doesn't use cascade delete."
- **No employee code (e.g. a short "007"-style reference) in the note yet** — `hr.employee.registration_number` ("Employee Reference") is the natural field for this, but it isn't populated for any employee yet. Once populated, one line of code adds it to the note.
- **An earlier, fully static per-model implementation** (`models/attribution_targets.py`, one `_inherit` class per model) exists in the codebase but is disabled (import commented out in `models/__init__.py`) — kept only as a fallback/reference, superseded by the two-layer `mail.thread` approach. Its dependency list (spanning many modules) is still declared in the manifest even though the file is disabled, so the fallback stays usable without a manifest change if ever re-enabled. Worth revisiting if the per-model approach is retired for good.
- **`ir.model` may still carry one orphaned, unused text column** (`employee_attribution_parent_field`, the pre-dropdown version of the parent-field setting) if `-u` was run while that version was briefly active — harmless, not yet cleaned up.

- **Two-factor auth coexistence:** if an account is ever both `is_shared_login` and has TOTP enabled, only the TOTP prompt fires today (not both in sequence). Not a concern currently since `auth_totp` isn't in use in this instance.
- **Coverage is bounded to messages that already get posted, for any model that hasn't had Layer 2 enabled.**
A model write that never triggers any message (no field tracking, no explicit log, Layer 2 off) has nothing to attribute — this is by design, not a gap, but worth knowing when auditing "why isn't this action showing an employee."

13. Full schema reference

All tables touched by this module, for reference:

TABLE	CHANGE
<code>res_users</code>	+ <code>is_shared_login</code> (bool)
<code>res_users_authorized_employee_rel</code>	new — plain M2M junction for <code>authorized_employee_ids</code> (who may bind to a shared account), not a log
<code>mail_message</code>	+ <code>employee_id</code> (FK <code>hr_employee</code>) — this is where every attribution note actually lives
<code>ir_model</code>	+ <code>employee_attribution_log_changes</code> (bool), + <code>employee_attribution_log_deletions</code> (bool), + <code>employee_attribution_parent_field_id</code> (FK <code>ir_model_fields</code>)
<code>hr_attribution_log</code>	new table (§6) — fallback for parentless deletions only

There is still no dedicated "all logs" table — querying "everything employee X did, across every model and every account" is `SELECT * FROM mail_message WHERE employee_id = X`, the same table Odoo's own chatter has always used, plus `hr_attribution_log` specifically for the deletion-with-no-parent case.

Internal implementation reference — Harleys Odoo 19 Enterprise. Click any screenshot to view it enlarged and step through the rest. Looking for a non-technical walkthrough instead? See the User Guide.